



PL-SQL

Lab -1

22AIE303 – Database Management Systems



NAME : JAYDEEP DILEEP
ROLL NO : AM.SC.U4AIE23114

1. Do all the questions discussed in the class

-- Write a function that finds the sum of 2 integers

create or replace function sum(a int, b int)

returns int **as**

\$\$

Declare

c int;

Begin

c:=a+b;

return c;

End;

\$\$

Language 'plpgsql';

-- Write a function that accepts a integer value and returns even or odd

create or replace function even_odd(a int)

returns varchar(10) **as**

\$\$

Begin

if a%2==0 **then**

return 'even';

else

return 'odd';

End if;

End;

\$\$

0 **Language** 'plpgsql';

create table Dept(dept_no int **primary key**,

dname varchar(10)

);

create table Emp(eno int **primary key**,

ename varchar(10),

sal numeric(10,2),

dno int **references** Dept(dept_no)

);

```

-- Write a function that accepts an employee name and returns the sal of the employee
create or replace function find_sal(empno emp.eno%type)
returns emp.sal%type as
$$
    Declare
        salary emp.sal%type;
    Begin
        select sal into salary from Emp where eno=empno;
        return salary;
    End;
$$
Language 'plpgsql';

```

```

-- Write a function that accepts dno and finds no of employees
create or replace function emp_count(deptno emp.dno%type)
returns int as
$$
    Declare
        num int;
    Begin
        select count(eno) into num from Emp where dno=deptno;
        return num;
    End;
$$
Language 'plpgsql';

```

```

-- Write a function eno and gives that employee 10% hike in salary if avg salary
-- of the corresponding department is greater than the employees current salary
create or replace function promo(empno emp.eno%type)
returns void as
$$
    Declare
        salary emp.sal%type;
        avg int;
    Begin
        select avg(sal) into avg from Emp where dno=(select dno from Emp where eno=empno);
        select sal into salary from Emp where eno=empno;
        if salary<avg then
            salary:=salary+(salary*0.1);
            update Emp set sal=salary where eno=empno;
        end if;
    End;
$$
Language 'plpgsql';

```

```

-- Write a function that accepts an employee name and returns the dep name of that employee
create or replace function emp_dep(empname emp.ename%type)
returns Dept.dname%type as
$$
    Declare
        dep Dept.dname%type;
    Begin
        select dname into dep from Dept where dept_no =(select dno from Emp where ename=empname);
        return dep;
    End;
$$
Language 'plpgsql';

```

2. Consider the following schema .

Emp(eno, ename, sal)

EMP_PROJ(eno, pno, prjt_hrs).

- a) Write a function ProjectLoad that returns the total project working hours for the given eno.
- b)
- c) Write a PL/SQL code to update the salary of an employee if the employee earn less than the average salary.

New salary is current sal + difference between current sal and average salary

```
create table emp_proj(eno int references Emp(eno),
                    pno int,
                    prjt_hrs numeric(6,2),
                    primary key (eno,pno)
);
```

-- 2.a.

```
create or replace function ProjectLoad(empno emp_proj.eno%type)
returns emp_proj.prjt_hrs%type as
$$
    Declare
        total emp_proj.prjt_hrs%type;
    Begin
        select sum(prjt_hrs) into total from emp_proj group by eno having eno=empno;
        return total;
    End;
$$
Language 'plpgsql';
```

-- 2.b.

```
create or replace function sal_update(empno emp.eno%type)
returns void as
$$
    Declare
        salary emp.sal%type;
        avg int;
    Begin
        select avg(sal) into avg from Emp where dno=(select dno from Emp where eno=empno);
        select sal into salary from Emp where eno=empno;
        if salary<avg then
            salary:=salary+(avg-salary);
            update Emp set sal=salary where eno=empno;
        end if;
    End;
$$
Language 'plpgsql';
```

3. Consider the following tables
Item(ino, iname, unit_price)
Transaction(tr_no,ino,qty)

Write a function to accept an item no from the user. If transaction has been made for less than 2 times for that item(check from transaction table), delete the item from the Item table.

```
create table Item(ino int primary key,
                 iname varchar(10),
                 unit_price numeric(8,2)
);

create table Transaction(tr_no int,
                        ino int references Item(ino),
                        qty int,
                        primary key(tr_no,ino)
);

create or replace function item_del(itno Item.ino%type)
returns void as
$$
    Declare
        amt int;
    Begin
        select count(tr_no) into amt from Transaction where ino = itno;
        if amt<2 then
            delete from Item where ino=itno;
        end if;
    End;
$$
Language 'plpgsql';
```

4.

Consider a Bank database which includes the following tables.

ACCOUNTS(ac_no,br_no, cust_no,ac_type,bal)
BRANCHES(br_no, br_name, loc)
CUSTOMER(cno, cname, c_type)

- Write a function that accepts a threshold value and a customer number .
The program updates the c_type based on the threshold value. Ie. If balance > threshold then class A, else class B.
- Write a function called CloseBranch that takes two arguments (the branch to be closed and the branch to take over the accounts) and transfers all accounts at the closing branch to the new branch and removes the closing branch.
- Write a function that implements a "safe" withdrawal operation, that only permits a withdraw if there sufficient funds in the account to cover it.

```
-- 4.
create table Branches(br_no int primary key,
                      br_name varchar(20),
                      loc varchar(50)
);

create table Customer(cno int primary key,
                     cname varchar(10),
                     c_type varchar(10)
);

create table Accounts(ac_no int primary key,
                     br_no int references Branches(br_no),
                     cust_no int references Customer(cno),
                     ac_type varchar(15),
                     bal numeric(10,2)
);

--4.a
create or replace function update_customer(threshold numeric, custno customer.cno%type)
returns void as
$$
    Declare
        total_bal numeric;
    Begin
        select sum(bal) into total_bal from accounts where cust_no = custno;
        if total_bal > threshold then
            update customer set c_type = 'A' where cno = custno;
        else
            update customer set c_type = 'B' where cno = custno;
        end if;
    End;
$$
Language plpgsql;

-- 4.b.
create or replace function CloseBranch(old_br branches.br_no%type, new_br branches.br_no%type)
returns void as
$$
    Begin
        update accounts set br_no = new_br where br_no = old_br;
        delete from branches where br_no = old_br;
    End;
$$
Language plpgsql;
```

```
-- 4.c.  
create or replace function SafeWithdraw(accno accounts.ac_no%type, amount numeric)  
returns varchar as  
$$  
    Declare  
        current_bal numeric;  
    Begin  
        select bal into current_bal from accounts where ac_no = accno;  
        if current_bal >= amount then  
            update accounts set bal = bal - amount where ac_no = accno;  
            return 'Withdrawal successful';  
        else  
            return 'Insufficient funds';  
        end if;  
    End;  
$$  
Language plpgsql;
```